HWK 2

Applied Natural Language Processing

DATA 641

Converting Jupyter Notebooks to HTML

To convert your Jupyter Notebook to an HTML file, follow these steps:

1. Open the Jupyter Notebook that you want to convert to an HTML file.
2. Ensure that the notebook is saved with the latest changes.
3. In the Jupyter Notebook interface, navigate to **File** in the menu bar.
4. Click on **Download as** and select **HTML (.html)** from the drop-down menu. This will initiate the conversion process.
5. The Jupyter Notebook will be converted to an HTML file and automatically downloaded to the default download location on your computer.
6. Locate the downloaded HTML file on your computer and verify that the conversion was successful.
7. Optionally, open the HTML file in a web browser to view and verify the content.
8. Upload your HTML file on Canvas.

Make sure to save your Jupyter Notebook before attempting to convert it to an HTML file.

Exercise 1 (15 points) Given a collection of COVID-19-related posts from various social media platforms our task will be to pre-process, and generate various types of features that can be used to train a model and classify a post as either *real* or *fake*.

(a) (0.5 points) Access and import the data from the TrainLabels.csv file.

(b) (0.5 points) For all posts within a category (fake or real), determine the total number of unique words, average words per post, and average characters per post. What do you observe about real news posts and fake news posts in terms of both characters per post and words per post?

(c) (10 points) For all posts use NLTK or any other NLP Python module and perform the following:

(i) Remove English stop words.

(ii) Remove urls from text and remove words starting with @ (mostly used for Twitter mentions) are removed. This is also accomplished via regular expressions.You can accomplish this via regular expressions[1].

[1] `https://stackoverflow.com/questions/24399820/expression-to-remove-url-links-from-twitter-tweet`

(iii) Lemmatization or stemming (Think what makes more sense for this dataset).

(iv) To account for differences in capitalization, transform each word to lowercase.

(v) Some social media posts contain XML entities such as "&amp". Convert XML entities into characters[2].

(e) (3 points) Extract features that reflect text understandability using the different readability metrics [3]. These features are based on properties such as words per sentence, characters per word, syllables per word, number of complex words, etc.

(f) (1 points) Calculate the average readability scores of the real post as well as the fake posts for each metric. What do you observe?

[2] https://stackoverflow.com/questions/701704/convert-html-entities-to-unicode-and-vice-versa

[3] https://pypi.org/project/textstat/